

observations. First, the denominator plays a uniquely critical role in sparse attention: an accurate estimate reduces bias in the stochastic error of the attention output. Unlike variance, bias compounds across sequence length, accelerating the accumulation of errors, thus degrading generation quality (See Appendix D.4 for proof). Second, the base sample size required to obtain estimates of statistics in case of denominator is significantly smaller than that needed for the numerator, making it computationally cheaper. Third, the denominator alone provides sufficient information to assign dynamic budgets across heads and queries and shows under this approximation the theoretical (ϵ, δ) values are strongly correlated (> 0.99 correlation) with observed errors (see Figure 1 and Appendix Figure 10 for more details.). For these reasons, our experiments employ this relaxation in all our experiments.

Implementation Details The index computation for vAttention is efficient even when the KV cache resides on the CPU. The computation is naturally vectorizable and well-suited for GPUs. Although vAttention requires partial access to the KV cache for budget calculations, this can be handled on the GPU using a small random cache that is incrementally populated and updated during token generation. When combined with approximate top- k methods such as HashAttention or DoubleSparsity, the auxiliary structures (e.g., bit cache or label cache) for top- k selection—orders of magnitude smaller than the full KV cache—can be stored directly on the GPU. For example, HashAttention requires only 32 bits per token per head for its bit cache.

5 EXPERIMENTS

Table 1: Average performance of different methods on RULER32K-HARD benchmark (consists of 7 datasets from RULER) at 10% sparsity. HashAttention is denoted as HAT. Detailed results are in Appendix A.3

	Llama-3.1-8B-Inst	Dpsk-R1-Distill-Llama-8B	Mistral-7B-Inst-v0.3
SDPA	88.74	65.41	64.05
oracle-top- k	87.18	64.87	64.37
vAttention(oracle-top- k)	88.61	65.15	64.12
HAT	81.94	60.70	54.66
vAttention(HAT)	86.56	65.06	56.90

We perform an elaborate evaluation of vAttention against oracle and approximate baselines on multiple datasets, models, and generation lengths. The evaluation setup is explained below.

Datasets and Models. We evaluate vAttention on four benchmark suites: RULER (32K context length) Hsieh et al. (2024), LongBench Bai et al. (2024), and Loogle (truncated to 16K) Li et al. (2023), providing a broad basis for comparison. We further extract seven tasks from RULER32K into RULER32K-HARD to isolate cases where top- k methods are known to struggle. RULER32K-HARD consists of `qa_1`, `qa_2`, `vt`, `fwe`, `niah_multikey_2`, `niah_multikey_3`, and `niah_multivalue`, selected based on the HashAttention paper, where these datasets were shown to be challenging. Detailed results are provided in the Appendix, with partial results included here. We also use AIME2024 to evaluate vAttention on long generation and reasoning tasks. For models, we consider three LLMs: Llama-3.1-8B-Instruct, Mistral-7B-v0.3, and DeepSeek-R1-Distill-Llama-8B, evaluated across different subsets of benchmarks.

Baselines. In our study, we choose *oracle-top- k* as a baseline, which serves as the theoretical gold standard for all approximate top- k methods, and *oracle-top- p* , the strongest oracle-top-based baseline, as it can provide a dynamic oracle-top- k based on the attention score distribution. As a representative of approximate top- k methods, we select HashAttention, which outperforms Quest Tang et al. (2024), Double Sparsity(Yang et al., 2024), InfLLM Xiao et al. (2024), and others. For completeness, a comparative table is provided in Appendix A.4. We report results for vAttention in combination with both oracle-top- k and HashAttention. When aiming to achieve a particular sparsity in an experiment, we search for the best parameters within a defined search space that yield the lowest local attention errors while meeting the target sparsity. Details of the search space are provided in Table 3. Following (Desai et al., 2025; Jegou et al., 2024; Hooper et al., 2024), we use full attention for context processing and sparse attention for question and generation. Under this setup, MagicPig does not perform well. We provide additional details in Appendix C.